

KRR Project — Milestone 3

From Ontology to R2RML Mapping

How the Ontology Drove Every Mapping Decision

ANKRI Mohamed-Khalil · MICHON Charlotte · PAULIS Antoine

Academic Year 2025–2026

1. Introduction — What is R2RML?

R2RML (Relational to RDF Mapping Language) is a W3C standard that defines how rows in a relational database are transformed into RDF triples. An R2RML mapping file is written in Turtle syntax and consists of a set of **TriplesMaps** — one per SQL table (or SQL query) that needs to be converted.

Each TriplesMap has three parts:

- **rr:logicalTable** — the SQL source: either a table name (rr:tableName) or a custom SQL query (rr:sqlQuery).
- **rr:subjectMap** — defines how to construct the IRI (the unique identifier) for each row, and which ontology class to assign to it via rr:class.
- **rr:predicateObjectMap** — defines one property to emit per map: the predicate (an ontology property) and its value (either a column value or a link to another TriplesMap via rr:parentTriplesMap).

The ontology drives every decision in the mapping file. The class URIs, property URIs, and IRI templates must all match **exactly** what is declared in the OWL file. A single character difference (e.g. uppercase vs lowercase) produces individuals that are invisible in Protege and produce no results in SPARQL queries.

2. Anatomy of a TriplesMap

The simplest possible TriplesMap is for a dictionary class like Genre. It reads the genre table, constructs an IRI for each row, declares the individual as `ont:Genre`, and emits a single data property.

```
<#MapGenre> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "genre" ] ;          # SQL source
  rr:subjectMap [
    rr:template "...resource/genre/{genre_id}" ;      # IRI template
    rr:class ont:Genre                                # rdf:type assertion
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:genreName ;                      # data property
    rr:objectMap [ rr:column "genre_name" ]           # column value
  ] .
```

This single TriplesMap produces two triples per row in the genre table:

- `res:genre/1` **rdf:type** `ont:Genre`
- `res:genre/1` **ont:genreName** `"Drama"`

2.1 The Three Types of objectMap

Type	Syntax	Used for
Column value	<code>rr:objectMap [rr:column "col_name"]</code>	Data properties — emits a literal string from a column
Direct template	<code>rr:objectMap [rr:template ".../{col}"]</code>	Object properties where the target has no TriplesMap (avoid — produces homeless individuals)
Join to parent	<code>rr:objectMap [rr:parentTriplesMap <#Map> ; rr:joinCondition [...]]</code>	Object properties — links two TriplesMaps via a SQL join condition

Note: In this project, direct templates were initially used for `ContentType` and `Region`. This produced homeless individuals (no `rdf:type`). The fix was to replace them with proper `joinCondition` references to dedicated TriplesMaps.

3. Mapping Dictionary Classes

Dictionary classes (Genre, Category, Role, ContentType, TitleType, Region) all follow the same pattern: one TriplesMap per table, one rr:class assertion, one data property for the name column. They are the simplest TriplesMaps in the file.

The critical requirement is that the class URI in rr:class and the property URI in rr:predicate must match **exactly** what the OWL file declares. Below is the complete list of dictionary TriplesMaps and their corresponding OWL declarations:

TriplesMap	SQL Table	rr:class	rr:predicate	rr:column
MapCategory	category	ont:Category	ont:categoryName	category_name
MapRole	role	ont:Role	ont:roleName	role_name
MapGenre	genre	ont:Genre	ont:genreName	genre_name
MapTitleType	title_type	ont:TitleType	ont:title_type_name	title_type_name
MapContentType	content_type	ont:ContentType	ont:content_type_name	content_type_name
MapRegion	region	ont:Region	ont:regionName	region_name
MapLanguage	language	ont:Language	(none declared in OWL)	—

Note: ont:title_type_name and ont:content_type_name use snake_case because the OWL file declared them with underscores. ont:categoryName, ont:roleName, and ont:genreName use camelCase. The mapping must match the OWL exactly — mixing conventions causes homeless individuals.

4. Mapping Core Entity Classes

4.1 MapTitle

The title table is the central entity. Its TriplesMap emits data properties and three object properties. The object property `ont:isClassifiedAs` is mapped via a `joinCondition` to `MapContentType`. The object property `ont:contains` is mapped via a `joinCondition` to `MapParticipation`. The object property `ont:isAka` is mapped via a `joinCondition` to `MapAlternativeTitle`.

```
<#MapTitle> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "title" ] ;
  rr:subjectMap [
    rr:template "...resource/title/{title_id}" ;
    rr:class ont:Title
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:primaryTitle ;
    rr:objectMap [ rr:column "primary_title" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:startYear ;
    rr:objectMap [ rr:column "start_year" ; rr:datatype xsd:string ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:isClassifiedAs ;
    rr:objectMap [
      rr:parentTriplesMap <#MapContentType> ;
      rr:joinCondition [ rr:child "content_type_id" ; rr:parent "content_type_id" ]
    ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:contains ;
    rr:objectMap [
      rr:parentTriplesMap <#MapParticipation> ;
      rr:joinCondition [ rr:child "title_id" ; rr:parent "title_id" ]
    ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:isAka ;
    rr:objectMap [
      rr:parentTriplesMap <#MapAlternativeTitle> ;
      rr:joinCondition [ rr:child "title_id" ; rr:parent "title_id" ]
    ]
  ] .
```

Note: Each `rr:joinCondition` specifies `rr:child` (the column in the current table) and `rr:parent` (the column in the target TriplesMap's table). The mapper performs a SQL JOIN internally to link the two individuals.

4.2 MapTalent

The talent table follows the same pattern. Its main object property is `ont:participatesIn`, which links each Talent to all its Participation individuals via a `joinCondition` on `talent_id`.

```
<#MapTalent> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "talent" ] ;
```

```

rr:subjectMap [
  rr:template "...resource/talent/{talent_id}" ;
  rr:class ont:Talent
] ;
rr:predicateObjectMap [
  rr:predicate ont:talentName ;
  rr:objectMap [ rr:column "talent_name" ]
] ;
rr:predicateObjectMap [
  rr:predicate ont:participatesIn ;
  rr:objectMap [
    rr:parentTriplesMap <#MapParticipation> ;
    rr:joinCondition [ rr:child "talent_id" ; rr:parent "talent_id" ]
  ]
] .

```

5. Mapping the Participation Bridge

The Participation TriplesMap is the most important in the file. It reads from `title_principal` and constructs a composite IRI from three columns: `title_id`, `talent_id`, and `ord`. The `ord` column ensures uniqueness when the same talent appears multiple times in the same title with different roles.

```
<#MapParticipation> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "title_principal" ] ;
  rr:subjectMap [
    rr:template "...resource/participation/{title_id}_{talent_id}_{ord}" ;
    rr:class ont:Participation
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:as ;
    rr:objectMap [
      rr:parentTriplesMap <#MapCategory> ;
      rr:joinCondition [ rr:child "category_id" ; rr:parent "category_id" ]
    ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:job ;
    rr:objectMap [ rr:column "job" ]
  ] .
```

5.1 The ont:plays Problem — talent_role has no title_id

The `ont:plays` property links a Participation to a Role. The role information is in the `talent_role` table, which has `talent_id` and `role_id` but **no title_id**. This means it is impossible to directly construct the Participation IRI from `talent_role` alone.

The solution is a dedicated TriplesMap that uses an **rr:sqlQuery** joining `title_principal` and `talent_role` on `talent_id`. This gives access to all three columns needed to build the Participation IRI (`title_id`, `talent_id`, `ord`) plus the `role_id` to link to the Role individual:

```
<#MapParticipationPlaysRole> a rr:TriplesMap ;
  rr:logicalTable [ rr:sqlQuery """
    SELECT tp.title_id, tp.talent_id, tp.ord, tr.role_id
    FROM title_principal tp
    JOIN talent_role tr ON tp.talent_id = tr.talent_id
    """ ] ;
  rr:subjectMap [
    rr:template "...resource/participation/{title_id}_{talent_id}_{ord}"
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:plays ;
    rr:objectMap [
      rr:parentTriplesMap <#MapRole> ;
      rr:joinCondition [ rr:child "role_id" ; rr:parent "role_id" ]
    ]
  ] .
```

The subject template in this TriplesMap is identical to the one in `MapParticipation`. This is intentional — it means the mapper will emit the `ont:plays` triple on the **same Participation individual** that was already

created by MapParticipation. The two TriplesMaps contribute different triples to the same node.

Note: A talent with 3 entries in talent_role will produce 3 ont:plays triples on the same Participation individual, correctly expressing that the participation involved multiple roles.

6. Mapping Junction Tables

Simple junction tables (`title_genre`, `title_aka_title_type`) do not create new individuals — they only add properties to existing ones. Their `TriplesMap` has a subject template that reconstructs the IRI of an already-existing individual, then adds a new property to it.

6.1 MapTitleGenre

The `title_genre` table links titles to genres. Its `TriplesMap` reconstructs the `Title` IRI from `title_id` and emits `ont:hasGenre` pointing to the `Genre` individual:

```
<#MapTitleGenre> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "title_genre" ] ;
  rr:subjectMap [
    rr:template "...resource/title/{title_id}" # no rr:class — Title already typed by
MapTitle
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:hasGenre ;
    rr:objectMap [
      rr:parentTriplesMap <#MapGenre> ;
      rr:joinCondition [ rr:child "genre_id" ; rr:parent "genre_id" ]
    ]
  ] .
```

Notice there is no `rr:class` in the `subjectMap`. The `rdf:type ont:Title` triple was already emitted by `MapTitle`. Adding it again would be harmless but redundant. Junction `TriplesMaps` only add properties — they do not re-declare types.

6.2 MapAkaTitleTypeLink

The `title_aka_title_type` table links alternative titles to their title types. It reconstructs the `AlternativeTitle` IRI using both `title_id` and `ord` (the same composite key used in `MapAlternativeTitle`):

```
<#MapAkaTitleTypeLink> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "title_aka_title_type" ] ;
  rr:subjectMap [
    rr:template "...resource/alternativeTitle/{title_id}_{ord}"
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:hasType ;
    rr:objectMap [
      rr:parentTriplesMap <#MapTitleType> ;
      rr:joinCondition [ rr:child "title_type_id" ; rr:parent "title_type_id" ]
    ]
  ] .
```

7. Mapping AlternativeTitle

The `title_aka` table maps to the `AlternativeTitle` class. It uses a composite IRI from `title_id` and `ord` (since the same title can have many alternative titles). The nullable `region_id` and `language_id` columns are handled via `joinConditions` — `NULL` values are silently skipped.

```
<#MapAlternativeTitle> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "title_aka" ] ;
  rr:subjectMap [
    rr:template "...resource/alternativeTitle/{title_id}_{ord}" ;
    rr:class ont:AlternativeTitle
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:alternativeTitleName ;
    rr:objectMap [ rr:column "aka_title" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ont:isFrom ;
    rr:objectMap [
      rr:parentTriplesMap <#MapRegion> ;
      rr:joinCondition [ rr:child "region_id" ; rr:parent "region_id" ] # NULL-safe
    ]
  ] .
```

Note: If a direct template had been used for `region_id` (`rr:template ".../region/{region_id}"`), a `NULL` value would produce the IRI `".../region/"` which is a malformed identifier. The `joinCondition` approach avoids this entirely.

8. IRI Design Principles

Every individual in the knowledge graph has a unique IRI constructed from a fixed base URI and a class-specific path segment followed by the primary key value(s). The base URI is:

```
http://www.semanticweb.org/antoi/data/resource/
```

The design follows these rules:

- **One path segment per class** — e.g. /title/, /talent/, /genre/ — so IRIs from different classes never collide.
- **Primary key appended directly** — e.g. /title/tt1234567, /genre/3.
- **Composite keys joined with underscore** — e.g. /participation/tt1234567_nm0000001_2 for the triple (title_id, talent_id, ord).
- **camelCase path segments** — e.g. /contentType/, /alternativeTitle/, /titleType/ to match OWL naming conventions.

Class	IRI Template	Key columns used
Title	.../title/{title_id}	title_id
Talent	.../talent/{talent_id}	talent_id
Participation	.../participation/{title_id}_{talent_id}_{ord}	title_id, talent_id, ord
AlternativeTitle	.../alternativeTitle/{title_id}_{ord}	title_id, ord
Genre	.../genre/{genre_id}	genre_id
Category	.../category/{category_id}	category_id
Role	.../role/{role_id}	role_id
ContentType	.../contentType/{content_type_id}	content_type_id
Region	.../region/{region_id}	region_id
TitleType	.../titleType/{title_type_id}	title_type_id

9. Common Mapping Errors and How We Fixed Them

This section documents the four concrete errors encountered during the mapping process, what caused them, and how they were resolved.

Error	Root Cause	Symptom	Fix
Homeless individuals for ContentType and Region	Used direct rr:template instead of rr:parentTriplesMap. IRI was minted but rdf:type was never asserted.	Protege shows 0 individuals under ContentType and Region classes.	Added MapContentType and MapRegion TriplesMaps. Replaced direct templates with joinCondition references.
ont:Role individuals exist but no Talent links to them	The talent_role junction table was never mapped at all.	Role class has 40 individuals but SPARQL queries on ont:plays return empty.	Added MapParticipationPlaysRole using rr:sqlQuery joining title_principal and talent_role.
Wrong predicate names (e.g. ont:jobDescription, ont:isFromRegion)	Property names in the mapping did not match the OWL exactly. OWL uses ont:job and ont:isFrom.	SPARQL queries on those properties return empty. Properties invisible in Protege.	Cross-referenced every predicate URI against the OWL file and corrected all mismatches.
NULL region_id causes broken IRIs	Direct template rr:template ".../region/{region_id}" with NULL value produces malformed IRI.	Some AlternativeTitle individuals have no isFrom triple; some mappers crash silently.	Switched to rr:joinCondition which skips NULL rows safely.

10. Complete TriplesMap Inventory

The final mapping file contains 12 TriplesMaps. The table below summarises each one with its SQL source, the ontology class or property it produces, and its type.

TriplesMap	SQL Source	Produces	Type
MapCategory	category	ont:Category individuals + ont:categoryName	Dictionary
MapRole	role	ont:Role individuals + ont:roleName	Dictionary
MapGenre	genre	ont:Genre individuals + ont:genreName	Dictionary
MapTitleType	title_type	ont:TitleType individuals + ont:title_type_name	Dictionary
MapContentType	content_type	ont:ContentType individuals + ont:content_type_name	Dictionary
MapRegion	region	ont:Region individuals + ont:regionName	Dictionary
MapLanguage	language	ont:Language individuals	Dictionary
MapTitle	title	ont:Title + primaryTitle, startYear, isClassifiedAs, contains, isAka	Core entity
MapTalent	talent	ont:Talent + talentName, participatesIn	Core entity
MapParticipation	title_principal	ont:Participation + ont:as, ont:job	Bridge
MapParticipationPlaysRole	title_principal JOIN talent_role	ont:plays on existing Participation	Bridge (SQL join)
MapTitleGenre	title_genre	ont:hasGenre on existing Title	Junction
MapAlternativeTitle	title_aka	ont:AlternativeTitle + alternativeTitleName, isFrom	Entity + join
MapAkaTitleTypeLink	title_aka_title_type	ont:hasType on existing AlternativeTitle	Junction

End of document.